

Document 13: Full MVP Build Order & Real Execution Plan

Project: Natural Intellects School Management System (NISMS)

Version: 1.0

Status: Execution Blueprint

Purpose:

This document defines the **exact build order** for turning all architecture, UI, and system design into a working product.

It removes confusion about “what to code first” and enforces a **real-world, dependency-safe development path**.

This is not theory.

This is the **actual implementation sequence**.

TABLE OF CONTENTS

1. Execution Philosophy
2. Build Dependency Logic
3. Phase 0: Project Setup
4. Phase 1: Core Infrastructure
5. Phase 2: Authentication & Roles
6. Phase 3: School Core System
7. Phase 4: Student Module
8. Phase 5: Teacher Module
9. Phase 6: Academic System
10. Phase 7: Finance System
11. Phase 8: Feature Flag System

- 12. Phase 9: Control Center (Admin SaaS Layer)
- 13. Phase 10: UI Integration
- 14. Phase 11: Testing & Hardening
- 15. Phase 12: Pilot Deployment
- 16. Critical Build Rules

1. EXECUTION PHILOSOPHY

Core Truth

You cannot build everything at once.

You must build in layers:

``text id="layer1" Foundation → Identity → Core Data → Features → Control Layer → UI → Deployment

```
---
```

```
## Rule
```

```
> If a system depends on another system, it must come AFTER it.
```

```
No exceptions.
```

```
---
```

```
# 2. BUILD DEPENDENCY LOGIC
```

```
---
```

```
## Dependency Tree
```

```
``text id="dep1"
```

```
Database
```

```
↓
```

```
Auth System
```

```
↓
```

```
School Core
```

```
↓
```

```
Users (Students/Teachers)
```

```
↓
```

```
Academics + Finance
```

↓
Feature Flags
↓
Control Center
↓
Full SaaS System

3. PHASE 0: PROJECT SETUP

Goal

Prepare clean development environment.

Tasks

- Initialize backend project (Node/NestJS or similar)
 - Initialize frontend project (Next.js or similar)
 - Setup database (PostgreSQL)
 - Setup ORM (Prisma recommended)
 - Setup Git repository
 - Setup environment variables system
-

Output

- Running empty system
 - Connected database
 - Basic project structure
-

4. PHASE 1: CORE INFRASTRUCTURE

Goal

Build system foundation.

Tasks

Database Core

- Base schema setup
 - Migrations system
 - Connection pooling
-

Logging System

- Error logging
 - System logs
 - Audit logs foundation
-

File Storage

- Image uploads
 - Document storage (student photos, receipts)
-

Output

- Stable backend foundation
 - File system ready
 - Logging active
-

5. PHASE 2: AUTHENTICATION & ROLES

Goal

Control access to everything.

Tasks

- Login system
 - JWT/session handling
 - Role-based access control (RBAC)
-

Roles

``text id="roles1" Super Admin School Admin Teacher Bursar Student Support Admin

Output

- Secure login system
- Role-based routing
- Protected APIs

6. PHASE 3: SCHOOL CORE SYSTEM

Goal

Create the "container" for everything.

Tasks

- School entity creation
- School settings
- School profile
- School subscription reference

Database Entities

- Schools
- SchoolSettings
- SchoolUsers

Output

- Multi-school support foundation
- School isolation logic

7. PHASE 4: STUDENT MODULE

Goal

Core education data system begins.

Tasks

- Student registration
- Student profiles
- Guardian data
- Student class assignment

Features

- Add student
- Edit student
- View student
- Archive student

Output

- Fully working student system

8. PHASE 5: TEACHER MODULE

Goal

Manage staff.

Tasks

- Teacher registration
- Subject assignment
- Class assignment

Output

- Teacher management system working

9. PHASE 6: ACADEMIC SYSTEM

Goal

Enable learning structure.

Tasks

- Classes
- Subjects
- Attendance system
- Marks entry
- Report generation

Output

- Full academic workflow functional

10. PHASE 7: FINANCE SYSTEM

Goal

Enable payments and revenue tracking.

Tasks

- Fee structures
- Payment recording
- Receipt generation
- Outstanding balances

Output

- Fully working school finance system

11. PHASE 8: FEATURE FLAG SYSTEM

Goal

Introduce SaaS flexibility.

Tasks

- Feature table creation
- Feature checks in backend
- Feature toggle logic
- Plan mapping system

Core Concept

```
```text id="feat1"
Feature exists once
Enabled per school
Controlled centrally
```

---

## Output

- Dynamic feature control system

---

## 12. PHASE 9: CONTROL CENTER (SAAS LAYER)

---

### Goal

Build Natural Intellects admin system.

---

### Tasks

- Schools dashboard
- Feature management UI
- Subscription control
- Billing tracking



- System analytics
- 

## Output

- Full SaaS control system
- 

## 13. PHASE 10: UI INTEGRATION

---

### Goal

Connect Figma design to real system.

---

### Tasks

- Build frontend pages
  - Connect APIs
  - Implement dashboards
  - Apply role-based UI rendering
  - Apply feature-based UI rendering
- 

## Output

- Fully functional UI system
- 

## 14. PHASE 11: TESTING & HARDENING

---

### Goal

Make system stable.

---

### Tasks

- Unit testing
  - Integration testing
  - Security testing
  - Performance testing
-

## Output

- Stable production-ready system
- 

## 15. PHASE 12: PILOT DEPLOYMENT

---

### Goal

Real-world validation.

---

### Tasks

- Deploy to first school
  - Monitor usage
  - Fix bugs
  - Collect feedback
- 

## Output

- Real operational system
- 

## 16. CRITICAL BUILD RULES

---

### Rule 1: Never Skip Authentication

Everything depends on identity.

---

### Rule 2: Build Backend Before UI

UI depends on APIs.

---

### Rule 3: Build Core Before SaaS Layer

Feature flags come AFTER working system.

---

## **Rule 4: Do Not Optimize Early**

First make it work. Then scale.

---

## **Rule 5: Control Center Comes LAST**

Because it manages everything else.

---

# **STRATEGIC CONCLUSION**

This build order ensures:

- No wasted development
- No broken dependencies
- Clean scaling path
- SaaS readiness from day one
- Controlled complexity growth

If followed strictly, Natural Intellects evolves from:

Simple school system

Into:

Fully scalable education SaaS platform

End of Document 13.